

MANUAL TECNICO

GalaPro - Sistema de Gestion de Clientes y Eventos

Estandar de Competencia:

EC0835 - Desarrollar software de aplicacion con acceso a bases de datos

Version:

1.0

Fecha:

Marzo 2026

Desarrollador:

Alexandro Reynoso

INDICE

1. Introduccion
2. Objetivo general del sistema
3. Objetivos especificos del sistema
4. Requerimientos de instalacion
5. Diagrama de clases
6. Diagrama entidad-relacion
7. Desarrollo (explicacion tecnica del funcionamiento)
- 8.Codigo de la aplicacion

1. INTRODUCCION

Este manual describe como esta construido por dentro el Sistema GalaPro. Es una aplicacion de escritorio desarrollada en Python para la agencia de eventos GalaPro, que permite gestionar clientes y sus eventos contratados mediante operaciones CRUD completas, reportes relacionales y un sistema de control de acceso con contraseñas cifradas.

El sistema se desarrollo siguiendo los criterios del estandar EC0835, que pide crear software con acceso a bases de datos relacionales usando orientacion a objetos, interfaz grafica y medidas de seguridad.

Este documento sirve para que cualquier persona que necesite entender, modificar o dar mantenimiento al sistema pueda hacerlo sin problemas.

2. OBJETIVO GENERAL DEL SISTEMA

Desarrollar una aplicacion de escritorio que permita a la agencia GalaPro registrar, organizar y consultar su cartera de

clientes y los eventos contratados por cada uno, implementando operaciones CRUD, reportes relacionales (JOIN y GROUP BY), control de acceso con hash de contraseñas y respaldo/restauración de la base de datos, todo desde una interfaz gráfica con tema oscuro construida con Python y Tkinter.

3. OBJETIVOS ESPECIFICOS DEL SISTEMA

- Implementar una base de datos SQLite con dos tablas relacionadas: clientes y eventos.
- Desarrollar operaciones CRUD completas para clientes y para eventos.
- Implementar validaciones obligatorias: código único por cliente, campos obligatorios y formato de fecha DD/MM/AAAA.
- Permitir actualizar el estatus de cada evento a: Cotizado, Confirmado, Realizado o Cancelado.
- Generar un reporte de combinación (JOIN) que muestre el detalle de cada evento junto con la información del cliente.
- Generar un reporte de agrupación (GROUP BY) que muestre la cantidad total de eventos por cliente.
- Implementar un sistema de login con contraseñas almacenadas como hash SHA-256.
- Implementar funciones de respaldo y restauración de la base de datos desde la interfaz gráfica.
- Compilar la aplicación en un archivo ejecutable .exe que funcione sin instalar Python.

4. REQUERIMIENTOS DE INSTALACION

4.1 Requerimientos de hardware

Componente	Minimo
Procesador	Intel Core i3 o equivalente
Memoria RAM	512 MB
Espacio en disco	50 MB
Pantalla	1024 x 768

4.2 Para ejecutar el .exe (usuario final)

Componente	Requerimiento
Sistema operativo	Windows 7, 8, 10 u 11
Dependencias	Ninguna

4.3 Para modificar el código fuente (desarrollador)

Componente	Version
Python	3.11.9
Tkinter	Incluido en Python
SQLite3	Incluido en Python
Hashlib	Incluido en Python

4.4 Pasos de instalacion del entorno

- 1. Instalar Python 3.11.9 desde python.org (marcar "Add Python to PATH").
- 2. Verificar en terminal: python --version
- 3. Instalar PyInstaller: pip install pyinstaller

4.5 Como ejecutar el sistema

Desde el ejecutable:

Abrir
GalaPro_Sistema.exe

Desde el codigo fuente:

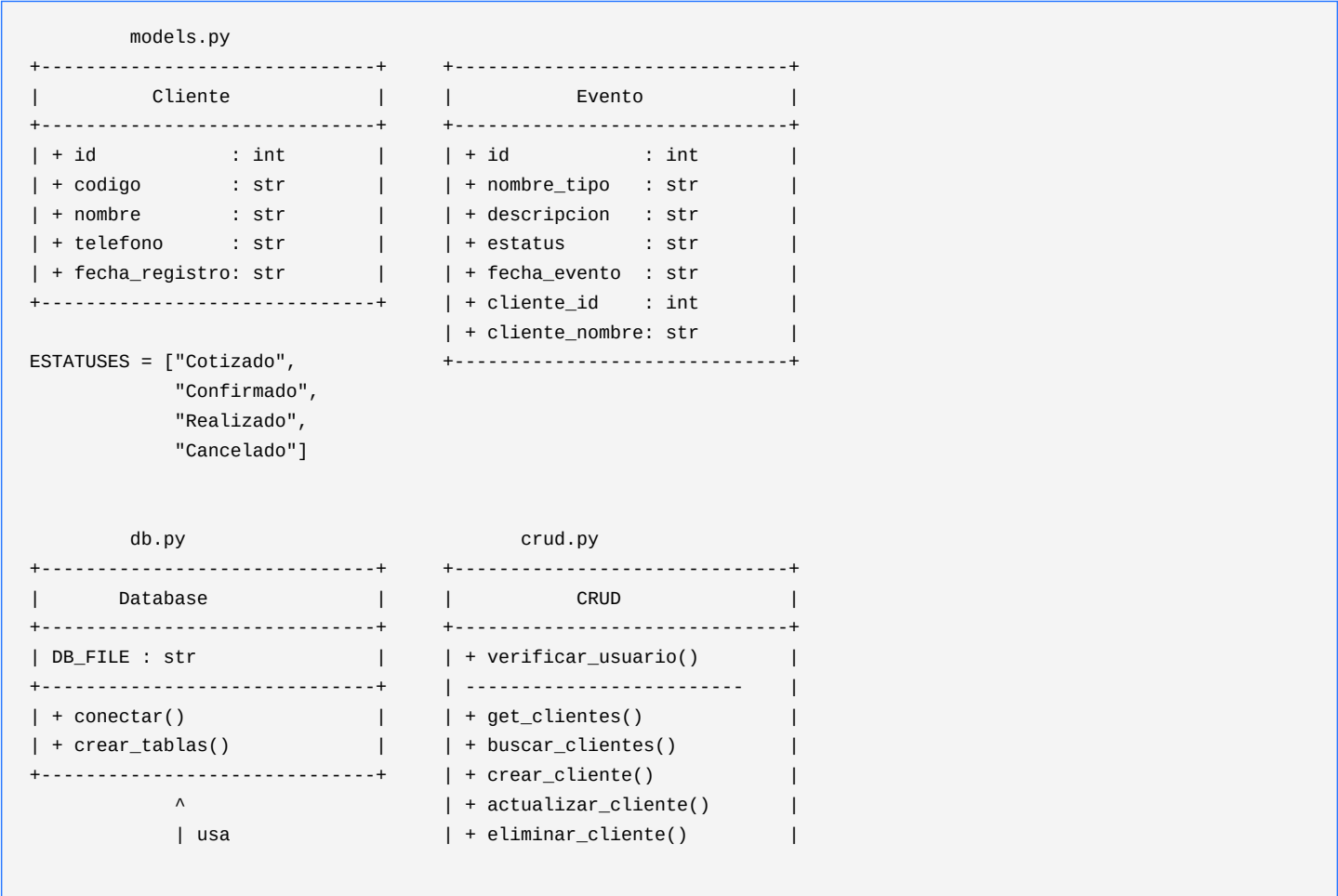
python main.py

Para recompilar el ejecutable:

```
pyinstaller GalaPro_Sistema.spec --distpath dist --workpath build --noconfirm
```

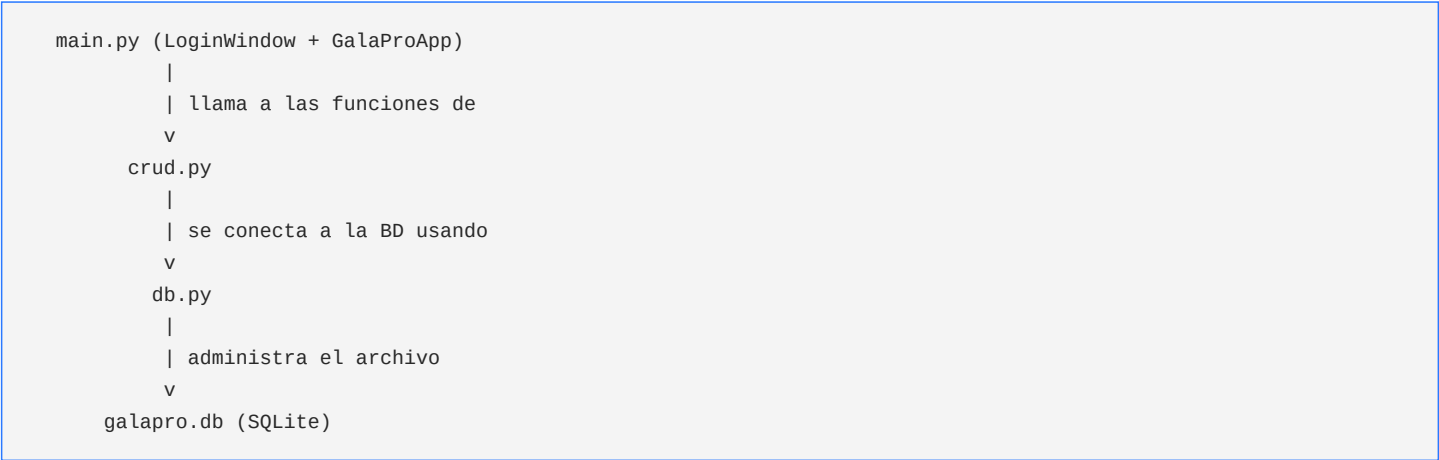
5. DIAGRAMA DE CLASES

El sistema tiene cuatro modulos principales. Aqui se muestra como esta organizado:



+-----+	+-----+
main.py	+ get_eventos()
+-----+	+ buscar_eventos()
LoginWindow	+ get_evento_por_id()
+ _build()	+ crear_evento()
+ _login()	+ actualizar_evento()
+-----+	+ actualizar_estatus_evento()
GalaProApp	+ eliminar_evento()
+ _tab_clientes()	-----
+ _tab_eventos()	+ reporte_join()
+ _tab_join()	+ reporte_groupby()
+ _tab_groupby()	+-----+
+ _respaldar()	
+ _restaurar()	
+-----+	

Como se conectan entre si



La interfaz grafica (main.py) llama a crud.py para hacer las operaciones, y crud.py usa db.py para conectarse a la base de datos SQLite.

6. DIAGRAMA ENTIDAD-RELACION

El sistema usa tres tablas en la base de datos SQLite.

Diagrama E-R

+-----+	+-----+
CLIENTES	EVENTOS
+-----+	+-----+
PK id INTEGER AUTO	PK id INTEGER AUTO
codigo TEXT UNIQUE	FK cliente_id INTEGER
nombre TEXT	nombre_tipo TEXT
telefono TEXT	descripcion TEXT
fecha_registro TEXT	estatus TEXT
+-----+	fecha_evento TEXT
	+-----+
+-----+	
USUARIOS	

```
+-----+
| PK  id   INTEGER AUTO |
|      usuario TEXT UNIQUE| (independiente - solo para login)
|      password_hash TEXT |
+-----+
```

Descripcion de los campos

Tabla: clientes

Campo	Tipo	Restriccion	Descripcion
id	INTEGER	PK AUTO	Identificador unico (automatico)
codigo	TEXT	NOT NULL, UNIQUE	Codigo interno del cliente (RFC)
nombre	TEXT	NOT NULL	Nombre completo del cliente
telefono	TEXT		Telefono de contacto
fecha_registro	TEXT	NOT NULL	Fecha de registro (DD/MM/AAAA)

Tabla: eventos

Campo	Tipo	Restriccion	Descripcion
id	INTEGER	PK AUTO	Identificador unico (automatico)
nombre_tipo	TEXT	NOT NULL	Nombre o tipo del evento
descripcion	TEXT		Descripcion de los requerimientos
estatus	TEXT	NOT NULL	Estado del evento
fecha_evento	TEXT	NOT NULL	Fecha del evento (DD/MM/AAAA)
cliente_id	INTEGER	FK NOT NULL	Referencia al cliente

Tabla: usuarios

Campo	Tipo	Restriccion	Descripcion
id	INTEGER	PK AUTO	Identificador unico (automatico)
usuario	TEXT	NOT NULL UNIQUE	Nombre de usuario para login
password_hash	TEXT	NOT NULL	Hash SHA-256 de la contraseña

Reglas de integridad

- clientes.codigo tiene restriccion UNIQUE: no se permiten dos clientes con el mismo codigo.
- eventos.cliente_id es llave foranea: todo evento debe pertenecer a un cliente existente.
- ON DELETE CASCADE: al eliminar un cliente se eliminan automaticamente todos sus eventos.
- Las contraseñas se almacenan como hash SHA-256, nunca en texto plano.

SQL de creacion de tablas

```
CREATE TABLE IF NOT EXISTS usuarios (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario     TEXT      NOT NULL UNIQUE,  
  password_hash TEXT     NOT NULL  
);
```

```

CREATE TABLE IF NOT EXISTS clientes (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo        TEXT      NOT NULL UNIQUE,
    nombre        TEXT      NOT NULL,
    telefono      TEXT,
    fecha_registro TEXT      NOT NULL
);

CREATE TABLE IF NOT EXISTS eventos (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre_tipo   TEXT      NOT NULL,
    descripcion    TEXT,
    estatus       TEXT      NOT NULL DEFAULT 'Cotizado',
    fecha_evento  TEXT      NOT NULL,
    cliente_id    INTEGER NOT NULL,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE CASCADE
);

```

7. DESARROLLO (EXPLICACION TECNICA DEL FUNCIONAMIENTO)

7.1 Estructura del proyecto

El proyecto esta dividido en 4 archivos de Python, cada uno con una funcion especifica:

```

Proyecto/
models.py ..... Define las clases Cliente y Evento
db.py ..... Maneja la conexion y creacion de la base de datos
crud.py ..... Operaciones CRUD, reportes y autenticacion
main.py ..... Interfaz grafica, login y ventana principal
galapro.db ..... La base de datos (se crea automaticamente)
GalaPro_Sistema.spec .. Configuracion de compilacion de PyInstaller
backup/ ..... Carpeta donde se guardan los respaldos
GalaPro_Sistema.exe ... El ejecutable compilado

```

7.2 models.py - Los modelos de datos

Define dos clases de datos:

- Cliente: Representa un cliente con los atributos id, codigo, nombre, telefono y fecha_registro.
- Evento: Representa un evento con los atributos id, nombre_tipo, descripcion, estatus, fecha_evento, cliente_id y cliente_nombre.
- ESTATUSSES: Lista con los cuatro estados validos de un evento: Cotizado, Confirmado, Realizado, Cancelado.

7.3 db.py - La base de datos

Maneja la conexion y estructura de la base de datos.

- _BASE: Detecta si el programa corre como .exe o como script para determinar la ruta correcta donde guardar la base de datos.
- DB_FILE: Ruta completa del archivo galapro.db.

- `conectar()`: Abre una conexion SQLite con timeout de 10 segundos, activa las llaves foraneas y usa modo WAL para evitar bloqueos.
- `crear_tablas()`: Crea las tres tablas si no existen y genera el usuario admin con su contraseña hasheada si no hay usuarios registrados.

7.4 crud.py - Las operaciones

Contiene todas las funciones que interactuan con la base de datos:

Autenticacion:

- `verificar_usuario(usuario, password)`: Calcula el hash SHA-256 de la contraseña y la compara con la almacenada en la base de datos.

Operaciones de Clientes:

- `get_clientes()`: Trae todos los clientes ordenados por ID.
- `buscar_clientes(termino)`: Busca clientes por codigo, nombre o telefono usando LIKE.
- `crear_cliente(...)`: Inserta un nuevo cliente. Lanza excepcion si el codigo ya existe.
- `actualizar_cliente(...)`: Modifica los datos de un cliente existente.
- `eliminar_cliente(id)`: Borra un cliente. Por CASCADE tambien elimina sus eventos.

Operaciones de Eventos:

- `get_eventos()`: Trae todos los eventos con JOIN a clientes para obtener el nombre del cliente.
- `buscar_eventos(termino)`: Busca por nombre_tipo, descripcion, estatus o nombre de cliente.
- `get_evento_por_id(id)`: Obtiene un evento especifico por su ID.
- `crear_evento(...)`: Inserta un nuevo evento asociado a un cliente.
- `actualizar_evento(...)`: Modifica todos los datos de un evento.
- `actualizar_estatus_evento(id, estatus)`: Solo actualiza el campo estatus de un evento.
- `eliminar_evento(id)`: Borra un evento especifico.

Reportes:

- `reporte_join()`: Consulta JOIN entre eventos y clientes, devuelve el detalle completo.
- `reporte_groupby()`: Consulta GROUP BY que cuenta eventos por cliente usando LEFT JOIN para incluir clientes sin eventos.

7.5 main.py - La interfaz grafica

Es la ventana principal del programa. Tiene dos clases:

LoginWindow:

- Ventana de inicio de sesion con fondo oscuro.
- Campos de usuario y contraseña.
- Al presionar Ingresar o Enter llama a `verificar_usuario()` de `crud.py`.
- Si las credenciales son incorrectas muestra un mensaje y limpia el campo de contraseña.

GalaProApp:

- Ventana principal con barra superior oscura y cuatro pestanas (Notebook).
- Pestana Clientes: formulario con 4 campos, botones CRUD, busqueda en tiempo real y tabla.
- Pestana Eventos: formulario con 6 campos (incluye combos para cliente, tipo y estatus), botones CRUD + Cambiar Estatus, busqueda y tabla.

- Pestana JOIN: tabla de solo lectura con el reporte combinado.
- Pestana GROUP BY: tabla de solo lectura con el conteo de eventos por cliente.
- Botones Respalidar BD y Restaurar BD en la barra superior.

Tema oscuro:

La funcion `aplicar_tema()` configura el estilo de todos los widgets ttk (Treeview, Entry, Combobox, LabelFrame, Notebook) con colores oscuros usando `ttk.Style` con el tema "clam" como base.

7.6 Como funciona cada operacion

Registrar un cliente:

El usuario llena el formulario, hace clic en "Agregar", el sistema valida que codigo y nombre no esten vacios y que la fecha tenga formato DD/MM/AAAA, luego inserta en la base de datos y actualiza la tabla.

Actualizar un cliente:

El usuario selecciona un cliente de la tabla (los datos se cargan en el formulario), modifica lo que necesita y hace clic en "Actualizar". Se validan los datos antes de guardar.

Eliminar un cliente:

El usuario selecciona un cliente y hace clic en "Eliminar". Se muestra una advertencia de que tambien se eliminaran sus eventos. Si confirma, se ejecuta DELETE en cascada.

Cambiar estatus de un evento:

El usuario selecciona un evento de la tabla, cambia el combo de Estatus al nuevo valor y hace clic en "Cambiar Estatus". Solo se actualiza ese campo sin tocar los demas datos.

Respalidar la base de datos:

Se usa `sqlite3.backup()` para copiar la base de datos activa a un archivo nuevo en la carpeta backup/ con timestamp en el nombre. Este metodo es seguro porque respeta las transacciones activas.

Restaurar la base de datos:

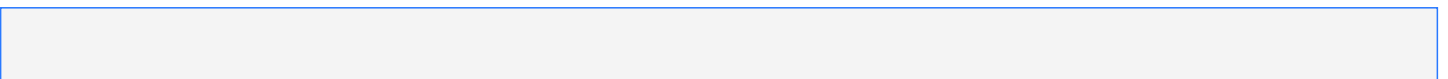
Se abre el explorador de archivos en la carpeta backup/. Al seleccionar un archivo y confirmar, se usa `sqlite3.backup()` para sobrescribir la base de datos actual y luego se refrescan todas las tablas de la interfaz automaticamente.

7.7 Validaciones y seguridad

- No se permite guardar un cliente sin codigo ni nombre.
- No se permite guardar un evento sin nombre/tipo.
- El codigo de cada cliente es unico (restriccion UNIQUE en la base de datos).
- Las fechas deben respetar el formato DD/MM/AAAA y ser fechas reales.
- Las contraseñas se almacenan como hash SHA-256 usando hashlib de Python.
- Las consultas SQL usan parametros con ? para prevenir inyeccion SQL.
- La base de datos usa PRAGMA foreign_keys = ON para respetar las relaciones.
- La conexion usa timeout=10 y journal_mode=WAL para evitar errores de bloqueo.

7.8 Compilacion a ejecutable

Se uso PyInstaller con el archivo de configuracion GalaPro_Sistema.spec:




```
pyinstaller GalaPro_Sistema.spec --distpath dist --workpath build --noconfirm
```

Opciones clave del .spec:

- console=False: Sin ventana de consola.
- upx=True: Compresion del ejecutable.
- El ejecutable resultante pesa aproximadamente 12 MB.

La ruta de la base de datos se calcula dinamicamente:

```
if getattr(sys, "frozen", False):
    _BASE = os.path.dirname(sys.executable) # junto al .exe
else:
    _BASE = os.path.dirname(os.path.abspath(__file__)) # junto al script
```

7.9 Pruebas realizadas

Prueba	Resultado
Login con credenciales correctas	Correcto
Login con credenciales incorrectas	Rechazado
Registrar cliente con datos validos	Correcto
Registrar cliente con codigo duplicado	Rechazado
Registrar cliente con campos obligatorios vacios	Rechazado
Registrar cliente con fecha invalida	Rechazado
Actualizar datos de un cliente	Correcto
Eliminar cliente (elimina eventos en cascada)	Correcto
Buscar clientes en tiempo real	Correcto
Registrar evento con datos validos	Correcto
Registrar evento con nombre/tipo vacio	Rechazado
Cambiar estatus de evento	Correcto
Reporte JOIN muestra cliente + evento	Correcto
Reporte GROUP BY muestra conteo por cliente	Correcto
Respaldar base de datos	Correcto
Restaurar base de datos y refrescar tablas	Correcto
Ejecutar el .exe sin Python instalado	Correcto
Base de datos se crea automaticamente	Correcto
Los datos se mantienen entre sesiones	Correcto

8. CODIGO DE LA APLICACION

A continuacion se presenta el codigo fuente completo de cada archivo.

8.1 models.py

```
class Cliente:
```

```

def __init__(self, id=None, codigo="", nombre="", telefono="", fecha_registro=""):
    self.id = id
    self.codigo = codigo
    self.nombre = nombre
    self.telefono = telefono
    self.fecha_registro = fecha_registro

class Evento:
    def __init__(self, id=None, nombre_tipo="", descripcion="", estatus="Cotizado",
                 fecha_evento="", cliente_id=None, cliente_nombre=""):
        self.id = id
        self.nombre_tipo = nombre_tipo
        self.descripcion = descripcion
        self.estatus = estatus
        self.fecha_evento = fecha_evento
        self.cliente_id = cliente_id
        self.cliente_nombre = cliente_nombre

ESTATUSES = ["Cotizado", "Confirmado", "Realizado", "Cancelado"]

```

8.2 db.py

```

import sqlite3
import hashlib
import os
import sys

if getattr(sys, "frozen", False):
    _BASE = os.path.dirname(sys.executable)
else:
    _BASE = os.path.dirname(os.path.abspath(__file__))

DB_FILE = os.path.join(_BASE, "galapro.db")

def conectar():
    conn = sqlite3.connect(DB_FILE, timeout=10)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    conn.execute("PRAGMA journal_mode=WAL")
    return conn

def crear_tablas():
    conn = conectar()
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS usuarios (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            usuario       TEXT    NOT NULL UNIQUE,
            password_hash TEXT    NOT NULL
        )
    """)

    cur.execute("""
        CREATE TABLE IF NOT EXISTS clientes (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            codigo        TEXT    NOT NULL UNIQUE,

```

```

        nombre          TEXT      NOT NULL,
        telefono        TEXT,
        fecha_registro TEXT      NOT NULL
    )
    """

cur.execute("""
CREATE TABLE IF NOT EXISTS eventos (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre_tipo TEXT      NOT NULL,
    descripcion TEXT,
    estatus     TEXT      NOT NULL DEFAULT 'Cotizado',
    fecha_evento TEXT      NOT NULL,
    cliente_id  INTEGER NOT NULL,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE CASCADE
)
""")

cur.execute("SELECT COUNT(*) FROM usuarios")
if cur.fetchone()[0] == 0:
    h = hashlib.sha256("admin123".encode()).hexdigest()
    cur.execute(
        "INSERT INTO usuarios (usuario, password_hash) VALUES (?, ?)",
        ("admin", h)
    )

conn.commit()
conn.close()

```

8.3 crud.py

```

import hashlib
from db import conectar
from models import Cliente, Evento

def verificar_usuario(usuario, password):
    h = hashlib.sha256(password.encode()).hexdigest()
    conn = conectar()
    cur = conn.cursor()
    cur.execute(
        "SELECT id FROM usuarios WHERE usuario = ? AND password_hash = ?",
        (usuario, h)
    )
    row = cur.fetchone()
    conn.close()
    return row is not None

def _fila_a_cliente(r):
    return Cliente(
        id=r["id"], codigo=r["codigo"], nombre=r["nombre"],
        telefono=r["telefono"] or "", fecha_registro=r["fecha_registro"]
    )

def get_clientes():
    conn = conectar()
    cur = conn.cursor()
    cur.execute("SELECT * FROM clientes ORDER BY id")
    rows = cur.fetchall()

```

```

conn.close()
return [_fila_a_cliente(r) for r in rows]

def buscar_clientes(termino):
    conn = conectar()
    cur = conn.cursor()
    t = f"%{termino}%"
    cur.execute(
        "SELECT * FROM clientes WHERE codigo LIKE ? OR nombre LIKE ? OR telefono LIKE ? ORDER BY id",
        (t, t, t)
    )
    rows = cur.fetchall()
    conn.close()
    return [_fila_a_cliente(r) for r in rows]

def crear_cliente(codigo, nombre, telefono, fecha_registro):
    conn = conectar()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO clientes (codigo, nombre, telefono, fecha_registro) VALUES (?, ?, ?, ?)",
        (codigo, nombre, telefono, fecha_registro)
    )
    conn.commit()
    conn.close()

def actualizar_cliente(id, codigo, nombre, telefono, fecha_registro):
    conn = conectar()
    cur = conn.cursor()
    cur.execute(
        "UPDATE clientes SET codigo=?, nombre=?, telefono=?, fecha_registro=? WHERE id=?",
        (codigo, nombre, telefono, fecha_registro, id)
    )
    conn.commit()
    conn.close()

def eliminar_cliente(id):
    conn = conectar()
    cur = conn.cursor()
    cur.execute("DELETE FROM clientes WHERE id=?", (id,))
    conn.commit()
    conn.close()

_SQL_EVENTOS = """
    SELECT e.id, e.nombre_tipo, e.descripcion, e.estatus, e.fecha_evento,
           e.cliente_id, c.nombre AS cliente_nombre
    FROM eventos e
    JOIN clientes c ON e.cliente_id = c.id
"""

def _fila_a_evento(r):
    return Evento(
        id=r["id"], nombre_tipo=r["nombre_tipo"],
        descripcion=r["descripcion"] or "", estatus=r["estatus"],
        fecha_evento=r["fecha_evento"], cliente_id=r["cliente_id"],
        cliente_nombre=r["cliente_nombre"]
    )

```

```

def get_eventos():
    conn = conectar()
    cur = conn.cursor()
    cur.execute(_SQL_EVENTOS + " ORDER BY e.id")
    rows = cur.fetchall()
    conn.close()
    return [_fila_a_evento(r) for r in rows]

def buscar_eventos(termino):
    conn = conectar()
    cur = conn.cursor()
    t = f"%{termino}%"
    cur.execute(
        _SQL_EVENTOS +
        " WHERE e.nombre_tipo LIKE ? OR e.descripcion LIKE ? OR e.estatus LIKE ? OR c.nombre LIKE ? ORDER BY e.id",
        (t, t, t, t)
    )
    rows = cur.fetchall()
    conn.close()
    return [_fila_a_evento(r) for r in rows]

def get_evento_por_id(id):
    conn = conectar()
    cur = conn.cursor()
    cur.execute(_SQL_EVENTOS + " WHERE e.id = ?", (id,))
    r = cur.fetchone()
    conn.close()
    return _fila_a_evento(r) if r else None

def crear_evento(nombre_tipo, descripcion, estatus, fecha_evento, cliente_id):
    conn = conectar()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO eventos (nombre_tipo, descripcion, estatus, fecha_evento, cliente_id) VALUES (?, ?, ?, ?, ?)",
        (nombre_tipo, descripcion, estatus, fecha_evento, cliente_id)
    )
    conn.commit()
    conn.close()

def actualizar_evento(id, nombre_tipo, descripcion, estatus, fecha_evento, cliente_id):
    conn = conectar()
    cur = conn.cursor()
    cur.execute(
        "UPDATE eventos SET nombre_tipo=?, descripcion=?, estatus=?, fecha_evento=?, cliente_id=? WHERE id=?",
        (nombre_tipo, descripcion, estatus, fecha_evento, cliente_id, id)
    )
    conn.commit()
    conn.close()

def actualizar_estatus_evento(id, estatus):
    conn = conectar()
    cur = conn.cursor()
    cur.execute("UPDATE eventos SET estatus=? WHERE id=?", (estatus, id))
    conn.commit()
    conn.close()

```

```
def eliminar_evento(id):
    conn = conectar()
    cur = conn.cursor()
    cur.execute("DELETE FROM eventos WHERE id=?", (id,))
    conn.commit()
    conn.close()

def reporte_join():
    conn = conectar()
    cur = conn.cursor()
    cur.execute("""
        SELECT e.id, c.codigo, c.nombre, e.nombre_tipo, e.descripcion, e.estatus, e.fecha_evento
        FROM eventos e
        JOIN clientes c ON e.cliente_id = c.id
        ORDER BY c.nombre, e.fecha_evento
    """)
    rows = cur.fetchall()
    conn.close()
    return [tuple(r) for r in rows]

def reporte_groupby():
    conn = conectar()
    cur = conn.cursor()
    cur.execute("""
        SELECT c.codigo, c.nombre, COUNT(e.id) AS total_eventos
        FROM clientes c
        LEFT JOIN eventos e ON c.id = e.cliente_id
        GROUP BY c.id, c.codigo, c.nombre
        ORDER BY total_eventos DESC, c.nombre
    """)
    rows = cur.fetchall()
    conn.close()
    return [tuple(r) for r in rows]
```

Fin del Manual Tecnico

Version: 1.0 | Fecha: Marzo 2026 | Desarrollador: Alexandro Reynoso